

Module 5: Code Optimization & Code Generation

Basic Blocks, CFG, DAG, Loop Optimizations, Data-Flow Equations & Register Coloring

Module V: Code Optimization & Target Code Generation – 12-Topic Master Guide

- Topic 36:** Issues in the Design of a Code Generator (Target Architecture)

Topic 38: Addresses in Target Code (Static & Dynamic Offset Calculations)

Topic 40: Control Flow Graphs (CFG Nodes, Edges, Dominators & Natural Loops)

Topic 42: Register Allocation & Assignment (Chaitin Graph Coloring Algorithm)

Topic 44: Data-Flow Analysis Frameworks (Reaching Definitions & Live Variables)

Topic 46: Peephole Optimization Techniques (Sliding Window Transformations)
- Topic 37:** Target Language Instruction Selection & Instruction Cost Models

Topic 39: Basic Blocks Partitioning (The 3 Classical Leader Rules)

Topic 41: Next-Use Information & Liveness Analysis Inside Blocks

Topic 43: Directed Acyclic Graphs (DAG) for Local Basic Block Optimization

Topic 45: Loop Optimizations (Loop-Invariant Code Motion & Strength Reduction)

Topic 47: Simple Code Generator Algorithm (Register & Address Descriptors)

Topic 36 – 38: Fundamental Issues in Target Code Generation

Design Challenge	Engineering Complexity & Tradeoffs	Compiler Strategy
1. Instruction Selection	Mapping uniform Three-Address Code into diverse, irregular target CPU instruction sets (CISC vs. RISC).	Tree-rewriting and dynamic programming instruction selection algorithms.
2. Register Allocation	Registers are the fastest hardware storage (1 cycle) but strictly limited in quantity (e.g., 16 or 32 physical registers).	NP-Complete Graph Coloring ; spills excess variables to stack frames.
3. Evaluation Order	The sequence in which computations execute dramatically alters the number of registers required.	Sethi-Ullman algorithm for optimal register labeling of syntax trees.

Topic 39 & 40: Basic Blocks & Control Flow Graphs (CFG)

A **Basic Block** is a maximal sequence of consecutive Three-Address instructions with a **single entry point** (execution enters only at the first instruction) and a **single exit point** (execution leaves only at the last instruction), with zero internal branches or jump targets.

The 3 Classical Rules for Leader Identification

An instruction *i* is designated a **Leader** if and only if it satisfies at least one condition:

- Rule 1:** The very first instruction of the Three-Address Code sequence is a Leader.
- Rule 2:** Any instruction that is the **target of a conditional or unconditional goto** is a Leader.
- Rule 3:** Any instruction that **immediately follows a conditional or unconditional goto** is a Leader.

Partitioning: A Basic Block starts at a Leader and extends up to, but not including, the next Leader or the end of the program.

Step-by-Step Solved Problem: Partitioning TAC into Basic Blocks & Constructing CFG

```
(1) i = 1 ; LEADER (Rule 1: First instruction)
(2) j = 1
(3) t1 = 10 * i ; LEADER (Rule 2: Target of goto 3 at line 11)
(4) t2 = t1 + j
(5) t3 = 8 * t2
(6) t4 = b[t3]
(7) a[t3] = t4
(8) j = j + 1
(9) if j ≤ 10 goto 3 ; Branch instruction
(10) i = i + 1 ; LEADER (Rule 3: Immediately follows goto 9)
(11) if i ≤ 10 goto 3 ; Branch instruction
(12) return ; LEADER (Rule 3: Immediately follows goto 11)
```

Identified Basic Blocks:

- B_1 : Instructions (1) to (2) [Initialization]
- B_2 : Instructions (3) to (9) [Inner loop body]
- B_3 : Instructions (10) to (11) [Outer loop step]
- B_4 : Instruction (12) [Exit]

Control Flow Graph Edges:

$$B_1 \longrightarrow B_2, \quad B_2 \xrightarrow{\text{true}} B_2, \quad B_2 \xrightarrow{\text{false}} B_3, \quad B_3 \xrightarrow{\text{true}} B_2, \quad B_3 \xrightarrow{\text{false}} B_4$$

Topic 42: Register Allocation via Graph Coloring (Chaitin-Briggs)

Kempe's Heuristic for K -Register Graph Coloring: 1. Construct **Interference Graph** $G = (V, E)$ where nodes represent variable live ranges, and edges connect variables simultaneously live at any program point. 2. **Simplify Step:** Find a node v with degree $< K$. Remove v from graph and push onto stack S . 3. If all nodes are removed, pop nodes from S one by one and assign available non-conflicting physical registers. 4. If a stage is reached where all remaining nodes have degree $\geq K$, select a variable to **Spill** to stack memory.

Topic 43: Directed Acyclic Graphs (DAG) for Local Optimization

A **Directed Acyclic Graph (DAG)** constructed for a single basic block exposes common subexpressions, eliminates redundant computations, and detects dead code locally.

Step-by-Step DAG Optimization Example

```
// Original Basic Block:
t1 = a + b
t2 = a + b      ; Redundant common subexpression!
t3 = t1 * t2
t4 = c + d
t5 = t3 + t4
```

DAG Construction: Identifies that `t1` and `t2` compute the identical node `+(a, b)`. Replaces `t2` with `t1`.

Optimized Generated Code:

```
t1 = a + b
t3 = t1 * t1      ; Reused t1!
t4 = c + d
t5 = t3 + t4
```

Topic 44 & 45: Principal Global & Loop Optimization Passes

Optimization Pass	Transformation Mechanism	Concrete Code Example
1. Constant Folding	Computes constant operations at compile time rather than generating runtime instructions.	<code>`x = 3.14159 * 2`</code> $\implies$ <code>`x = 6.28318`</code>
2. Constant Propagation	Replaces variable uses with known constant values throughout reaching definitions.	<code>`x = 10; y = x + 5;`</code> $\implies$ <code>`y = 10 + 5;`</code> $\implies$ <code>`y = 15;`</code>
3. Loop-Invariant Code Motion	Hoists statements whose operands do not change during loop iterations into a loop pre-header.	<code>`while (i < N) { x = a + b; ... }`</code> $\implies$ <code>`x = a + b; while (i < N) { ... }`</code>
4. Strength Reduction	Replaces expensive operations (multiplication/division) with equivalent cheaper operations (addition/shifts).	<code>`for (i=0; i</code>
5. Dead Code Elimination	Removes statements computing values that are never subsequently read on any execution path.	<code>`if (0) { foo(); }`</code> is entirely deleted from the binary.

Topic 46: Peephole Optimization Techniques

The 4 Classical Peephole Transformations

Operates on a small sliding window (typically 2–4 instructions) of target assembly code:

1. **Eliminating Redundant Load/Store:** ``MOV RO, a`` immediately followed by ``MOV a, RO`` (second instruction is deleted).
2. **Eliminating Unreachable Code:** Instructions immediately following an unconditional ``JMP`` that lack a jump label are deleted.
3. **Algebraic Simplifications:** Replaces ``x = x + 0`` or ``x = x * 1`` with no-ops; replaces ``x = x * 2`` with fast shift ``SHL RO, 1``.
4. **Machine Idioms:** Replaces ``MOV RO, #0`` with single-cycle ``XOR RO, RO``.

Topic 47: Simple Code Generation Algorithm

Register Descriptors & Address Descriptors: – **Register Descriptor (RD[R]):** Tracks all variables currently held inside physical register R . – **Address Descriptor (AD[v]):** Tracks all memory and register locations where current value of variable v can be found.

M5 Active Recall & 10–Question University Exam Master Bank

Q1. Explain the Data–Flow Analysis framework for Reaching Definitions with mathematical equations. (10 Marks)

A definition $d : u = v + w$ **reaches** a point p if there exists an execution path from d to p along which d is not "killed" (redefined).

Data–Flow Transfer Equations:

$$\text{IN}[\mathbf{B}] = \bigcup_{\mathbf{P} \in \text{Pred}[\mathbf{B}]} \text{OUT}[\mathbf{P}]$$
$$\text{OUT}[\mathbf{B}] = \text{GEN}[\mathbf{B}] \cup (\text{IN}[\mathbf{B}] - \text{KILL}[\mathbf{B}])$$

Where $\text{GEN}[\mathbf{B}]$ is the set of definitions generated inside block $\mathbf{B}$, and $\text{KILL}[\mathbf{B}]$ is the set of all definitions elsewhere in the program that define the same variables modified in $\mathbf{B}$. Solved iteratively to reach maximum fixed–point convergence!

Q2. State the 3 Leader Identification Rules for partitioning Three–Address Code into Basic Blocks. (6 Marks)

1. **Rule 1:** The very first instruction of the Three–Address Code sequence is a Leader.
2. **Rule 2:** Any instruction that is the target of a conditional or unconditional branch is a Leader.
3. **Rule 3:** Any instruction that immediately follows a conditional or unconditional branch is a Leader.

Q3. Explain Loop–Invariant Code Motion and Induction Variable Elimination with concrete code examples. (8 Marks)

- **Loop–Invariant Code Motion (Hoisting):** Computations inside a loop whose operands never change across iterations are moved outside the loop into a newly created pre–header block.
- **Induction Variable Elimination:** When two variables i and j change in lockstep inside a loop (e.g., $j = 4 * i$), j can be updated via addition ($j += 4$) and i can be eliminated entirely if it is not used outside the loop.

Q4. What is Peephole Optimization? Explain 4 classical peephole optimization techniques. (8 Marks)

Peephole optimization is a local target–code optimization technique that inspects a short sliding window of target instructions (2 to 4 instructions) to perform immediate peephole replacements:

1. **Redundant Load/Store Elimination:** Deleting redundant `MOV x, R0` immediately after `MOV R0, x`.
2. **Unreachable Code Elimination:** Deleting instructions following an unconditional `JMP` that lack a target label.
3. **Flow of Control Optimization:** Replacing jumps to jumps (`JMP L1` where `L1: JMP L2`) with `JMP L2`.
4. **Algebraic Simplification & Strength Reduction:** Replacing $x = x * 2$ with single–cycle `SHL R0, 1`.

Q5. Explain the Chaitin-Briggs Graph Coloring algorithm for Register Allocation. (10 Marks)

Register allocation is formulated as graph K -coloring where vertices represent variable live ranges and edges represent simultaneous liveness (interference):

1. **Build:** Construct interference graph $G = (V, E)$.
2. **Simplify (Kempe's Heuristic):** Find vertex v with degree $< K$. Remove v and push onto stack S .
3. **Spill:** If all remaining vertices have degree $\geq K$, select a candidate with high spill cost / low frequency to spill to memory.
4. **Select:** Pop vertices from S and assign non-conflicting colors (registers). If spilled, insert load/store instructions and re-run!

Topic 47.2: Comprehensive Data-Flow Analysis & Optimization Traces

🏠 Solved Problem 1: Reaching Definitions Iterative Fixed-Point Table Convergence

Consider Control Flow Graph with 3 Basic Blocks:

- $B_1: d_1 : i = 1, d_2 : j = 1 \implies \text{GEN}[B_1] = \{d_1, d_2\}, \text{KILL}[B_1] = \{d_3, d_4\}$
- $B_2: d_3 : i = i + 1, d_4 : j = j + 1 \implies \text{GEN}[B_2] = \{d_3, d_4\}, \text{KILL}[B_2] = \{d_1, d_2\}$
- $B_3: d_5 : x = i + j \implies \text{GEN}[B_3] = \{d_5\}, \text{KILL}[B_3] = \emptyset$

Iterative Fixed-Point Convergence Table:

Iteration	IN[B_1]	OUT[B_1]	IN[B_2]	OUT[B_2]	IN[B_3]	OUT[B_3]
Init	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
Pass 1	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$
Pass 2	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2, d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$
Pass 3 (Converged)	$\emptyset$	$\{d_1, d_2\}$	$\{d_1, d_2, d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4\}$	$\{d_3, d_4, d_5\}$

🏠 Solved Problem 2: Loop-Invariant Code Motion & Induction Variable Strength Reduction

Unoptimized Source Loop:

```
for (i = 0; i < N; i++) {
    x = a + b;           // Loop Invariant!
    arr[i] = i * 4;      // Induction Variable with multiplication!
}
```

Optimized Generated Code:

```
x = a + b;              // Hoisted to loop pre-header
p = &arr[0];            // Pointer initialization
for (i = 0; i < N; i++) {
    *p = i << 2;         // Strength reduction: replaced mult with fast shift & pointer addition
    p++;
}
```

Q6. Compare Local Optimization, Loop Optimization, and Global Optimization across scope and algorithms. (8 Marks)

- **Local Optimization:** Operates strictly within a single Basic Block using Directed Acyclic Graphs (DAG) for common subexpression elimination, algebraic identities, and dead code elimination.
- **Loop Optimization:** Operates on natural loops in the CFG using Loop-Invariant Code Motion (hoisting statements to loop pre-headers), Induction Variable Elimination, and Strength Reduction.
- **Global Optimization:** Operates across multiple basic blocks throughout the entire CFG using Iterative Data-Flow Analysis (Reaching Definitions, Available Expressions, Live Variable Analysis).

Q7. Explain Dominators, Immediate Dominators, and Natural Loops in Control Flow Graphs. (10 Marks)

- **Dominator ($d \text{ dom } n$):** Node d dominates node n if every execution path from the CFG entry node to n must pass through d .
- **Immediate Dominator ($\text{idom}(n)$):** The unique dominator d of n ($d \neq n$) that does not strictly dominate any other strict dominator of n . Forming the **Dominator Tree**.
- **Natural Loop:** A loop characterized by a single loop header d and a **Back-Edge** $n \rightarrow d$ where $d \text{ dom } n$. The natural loop consists of d plus all nodes that can reach n without passing through d .

Q8. Explain Available Expressions Data-Flow Analysis and Global Common Subexpression Elimination. (8 Marks)

An expression $x + y$ is **Available** at point p if every path from the initial node to p evaluates $x + y$, and neither x nor y is subsequently redefined.

Transfer Equations (Must-Analysis / Intersection Meet):

$$\text{IN}[\mathbf{B}] = \bigcap_{\mathbf{P} \in \text{Pred}[\mathbf{B}]} \text{OUT}[\mathbf{P}]$$
$$\text{OUT}[\mathbf{B}] = \text{GEN}[\mathbf{B}] \cup (\text{IN}[\mathbf{B}] - \text{KILL}[\mathbf{B}])$$

Used to eliminate redundant evaluations globally across the entire CFG.

Q9. Explain Live Variable Analysis and its role in Dead Code Elimination and Register Allocation. (8 Marks)

A variable v is **Live** at point p if there exists an execution path from p along which v is read before any subsequent redefinition.

Transfer Equations (Backward Data-Flow Analysis):

$$\text{OUT}[\mathbf{B}] = \bigcup_{\mathbf{S} \in \text{Succ}[\mathbf{B}]} \text{IN}[\mathbf{S}]$$
$$\text{IN}[\mathbf{B}] = \text{DEF}[\mathbf{B}] \cup (\text{OUT}[\mathbf{B}] - \text{USE}[\mathbf{B}])$$

Variables not live after a statement are dead (allowing assignment removal), and simultaneously live variables define edges in the Register Interference Graph.

Q10. Explain the Simple Code Generator algorithm with Register Descriptors and Address Descriptors. (10 Marks)

The simple code generator processes Three-Address instructions $x = y \text{ op } z$ sequentially within a basic block:

1. Invokes **getReg**($x = y \text{ op } z$) to determine the optimal physical register R_y for operand y .
2. If y is not already in R_y , emits `MOV R\_y, y`.
3. Emits `OP R\_y, z` (where z is in a register or memory location).
4. Updates **RD**[R_y] to indicate R_y now contains variable x .
5. Updates **AD**[x] to indicate x resides in register R_y , and removes x from other descriptors.

Q11. Explain Constant Folding vs. Constant Propagation with before and after code traces. (6 Marks)

- **Constant Folding:** Evaluates arithmetic operations on constant literals at compile-time: ``x = 10 * 20`  $\implies$  `x = 200`.`
- **Constant Propagation:** Replaces variable uses with their statically known constant values throughout the CFG: ``x = 10; y = x + 5;`  $\implies$  `y = 10 + 5;`  $\implies$  `y = 15;`.`

Q12. Explain the Sethi-Ullman Algorithm for Optimal Instruction Scheduling and Register Labeling. (8 Marks)

The **Sethi-Ullman Algorithm** computes the minimum number of registers required to evaluate an expression tree without spilling to memory:

1. Leaves (variables) are assigned label ``1``; right leaves of binary operations are assigned ``0``.
2. For a binary node with left child label l_1 and right child label l_2 :

$$\text{Label}(\text{Node}) = \begin{cases} \max(l_1, l_2) & \text{if } l_1 \neq l_2 \\ l_1 + 1 & \text{if } l_1 = l_2 \end{cases}$$

3. The compiler generates code evaluating the child subtree with the higher label first, minimizing temporary register holding time!

Topic 47.3: Advanced Register Allocation & Global Data-Flow Frameworks

Register allocation is widely considered one of the most critical optimization passes in modern compilers. Because memory access takes 100–300 CPU clock cycles while register access takes only 1 cycle, maximizing variable residence in registers yields order-of-magnitude execution speedups.

Complete Step-by-Step Chaitin-Briggs Graph Coloring Trace

Let variables A, B, C, D, E have the following Interference Graph edges for a target CPU with $K = 2$ physical registers (R1, R2):

- A interferes with $\{B, C\}$ (Degree 2)
- B interferes with $\{A, C, D\}$ (Degree 3)
- C interferes with $\{A, B, E\}$ (Degree 3)
- D interferes with $\{B\}$ (Degree 1)
- E interferes with $\{C\}$ (Degree 1)

Graph Coloring Execution Trace:

1. **Step 1:** Node D has degree $1 < 2$. Remove D and push onto stack $S = [D]$.
2. **Step 2:** Node E has degree $1 < 2$. Remove E and push onto stack $S = [D, E]$.
3. **Step 3:** Remaining graph nodes: A (degree 2), B (degree 2), C (degree 2). Since all degrees are ≥ 2 , Kempe's heuristic selects A to remove: Stack $S = [D, E, A]$.
4. **Step 4:** Remaining nodes B and C now have degree 1. Remove B : Stack $S = [D, E, A, B]$; Remove C : Stack $S = [D, E, A, B, C]$.
5. **Reconstruction & Color Assignment:**
 - Pop C : Assign Register ****R1****.
 - Pop B : Assign Register ****R2**** (since B interferes with C).
 - Pop A : Interferes with B (R2) and C (R1) $\implies$ **Spill A to Stack Memory!**
 - Pop E : Interferes only with C (R1) $\implies$ Assign Register ****R2****.
 - Pop D : Interferes only with B (R2) $\implies$ Assign Register ****R1****.

🏛️ Step-by-Step Dominator Tree & Natural Loop Identification Algorithm

Consider CFG nodes $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ (Back-edge $4 \rightarrow 2$):

- Node 1 dominates $\{1, 2, 3, 4\}$
- Node 2 dominates $\{2, 3, 4\}$
- Node 3 dominates $\{3, 4\}$
- Node 4 dominates $\{4\}$

Because 2 **dom** 4, the edge $4 \rightarrow 2$ is a **Back-Edge**. The Natural Loop consists of header node 2 plus all nodes that can reach node 4 without passing through 2, yielding loop node set $\{2, 3, 4\}$.

Topic 47.4: Advanced Instruction Selection & Available Expressions Worklist

Instruction Selection maps the machine-independent Three-Address intermediate code into concrete target processor instructions. The compiler models target CPU instructions as **Tree Patterns (Tiles)** and tiles the intermediate representation Abstract Syntax Tree with minimum cost.

Instruction Selection Technique	Operating Algorithm & Tree Tiling Strategy	Computational Complexity
1. Maximal Munch Algorithm	Greedy top-down pattern matcher: at root of tree, finds the largest matching tile covering the most nodes, emits instruction, and recurses on subtrees.	Linear Time $O(N)$; produces high-quality RISC and CISC instructions.
2. Dynamic Programming Tiling	Bottom-up optimal tiling: computes minimum cost to evaluate each subtree in each hardware register class, then traverses downward to emit instructions.	Optimal Code in $O(N \cdot P)$ time, where $ P $ is the number of target instruction patterns.
3. Tree-Rewriting / BURG Systems	Uses Tree Grammars and Bottom-Up Rewrite Generators to automatically generate table-driven optimal instruction selectors.	Industry standard in production compilers (GCC, LLVM SelectionDAG).

🏛️ Step-by-Step Solved Problem: Iterative Available Expressions Data-Flow Convergence

Consider a CFG with 3 basic blocks and expression set $U = \{a + b, c * d, e - f\}$:

- B_1 : Computes $a + b$, defines $x \implies \text{GEN}[B_1] = \{a + b\}, \text{KILL}[B_1] = \emptyset$
- B_2 : Computes $c * d$, redefines $a \implies \text{GEN}[B_2] = \{c * d\}, \text{KILL}[B_2] = \{a + b\}$
- B_3 : Computes $e - f \implies \text{GEN}[B_3] = \{e - f\}, \text{KILL}[B_3] = \emptyset$

Available Expressions Iterative Convergence Table (Must-Analysis: $\text{IN}[B] = \bigcap \text{OUT}[P]$):

Iteration	$\text{IN}[B_1]$	$\text{OUT}[B_1]$	$\text{IN}[B_2]$	$\text{OUT}[B_2]$	$\text{IN}[B_3]$	$\text{OUT}[B_3]$
Init	$\emptyset$	U	U	U	U	U
Pass 1	$\emptyset$	$\{a + b\}$	$\{a + b\}$	$\{c * d\}$	$\{c * d\}$	$\{c * d, e - f\}$
Pass 2 (Converged)	$\emptyset$	$\{a + b\}$	$\{a + b\}$	$\{c * d\}$	$\{c * d\}$	$\{c * d, e - f\}$

Solved Problem: DAG Construction for 8-Statement Basic Block

Basic Block Statements:

```
(1) a = b + c
(2) d = b + c      ; Common Subexpression: Reuses DAG node +(b, c)
(3) e = a + d
(4) f = b + c      ; Common Subexpression: Reuses DAG node +(b, c)
(5) g = e * f
(6) h = a + d      ; Common Subexpression: Reuses DAG node +(a, d)
(7) i = g - h
(8) x = i
```

Optimized Target Code Generated from DAG:

```
a = b + c
d = a          ; Copy instead of addition
e = a + a
g = e * a
h = e          ; Copy instead of addition
x = g - e      ; Direct assignment eliminating temporaries f, h, i
```

Topic 47.5: Interprocedural Optimization (IPO) & Whole-Program Compilation

While intraprocedural optimization operates within a single procedure, **Interprocedural Optimization (IPO)** analyzes relationships across the entire call graph of the program.

Interprocedural Pass	Operating Mechanism	Optimization Impact & Tradeoffs
1. Function Inlining	Replaces a procedure call site with the verbatim body of the callee, substituting actual parameters for formal variables.	Eliminates function call overhead (10 to 30 cycles) and enables massive downstream constant propagation; may increase code size.
2. Interprocedural Constant Propagation (IPCP)	Tracks constant values passed as arguments across procedure boundaries throughout the Call Graph.	Specializes procedures for common constant inputs and prunes dead branches globally.
3. Devirtualization	Proves via Class Hierarchy Analysis (CHA) that a virtual call site has only one possible target subclass.	Converts expensive indirect virtual dispatch calls into direct static calls or inlined code!

Step-by-Step Solved Problem: Function Inlining & Downstream Optimization Cascade

Original Code:

```
int square(int val) { return val * val; }
int compute() {
    int x = 5;
    return square(x) + 10;
}
```

Optimization Cascade Trace:

1. **Pass 1 (Function Inlining):** Replaces `square(x)` with `(x * x)` $\implies$ `return (x * x) + 10;`
2. **Pass 2 (Constant Propagation):** Replaces `x` with `5` $\implies$ `return (5 * 5) + 10;`
3. **Pass 3 (Constant Folding):** Evaluates `5 * 5 = 25` and `25 + 10 = 35` at compile time.
4. **Final Generated Target Code:** `MOV EAX, 35; RET;` (Zero memory access, zero multiplication, instant return!)